

```
1  /*
2
3  --> Map interface:
4
5  • It was introduced in JDK 1.2 version.
6  • It is not child interface to Collection Interface.
7  • It able to arrange all the elements in the form of Key-value pairs.
8  • In Map, both keys and values are objects.
9  • Duplicates are not allowed at keys, but values may be duplicated.
10 • Only one null value is allowed at keys side, but, any number of null
    values are allowed at values side.
11 • Both keys and Values are able to allow heterogeneous elements.
12 • Insertion order is not followed.
13 • Sorting order is not followed.
14
15 --> HashMap:
16
17 • It was introduced in JDK1.2 version.
18 • It is not Legacy.
19 • It is an implementation class to Map interface.
20 • It able to arrange all the elements in the form of Key-value pairs.
21 • In HashMap, both keys and values are objects.
22 • Duplicates are not allowed at keys, but values may be duplicated.
23 • Only one null value is allowed at keys side, but, any number of null
    values are allowed at values side.
24 • Both keys and Values are able to allow heterogeneous elements.
25 • Insertion order is not followed.
26 • Sorting order is not followed.
27 • Its internal data structure is "Hashtable".
28 • Its initial capacity is 16 elements.
29 • It is not synchronized.
30 • No method is synchronized in HashMap.
31 • It allows more than one thread to access data.
32 • It follows parallel execution.
33 • It will reduce application execution time.
34 • It will improve application performance.
35 • It is not giving guarantee for data consistency.
36 • It is not threadsafe.
37
38 --> Constructors:
39 1. public HashMap()
40 2. public HashMap(int capacity)
41 3. public HashMap(int capacity, float fill_Ratio)
42 4. public HashMap(Map m)
43
44 * Methods:
45
46 1. public void put(Object key, Object value):
47    --> It will add the specified key-value pair to Map.
48
49 2. public void putAll(Map m):
50    --> It will add all key-value pairs of the specified map to the
    present Map object.
51
52 3. public Object get(Object key):
```

```

53     --> It will return value of the specified key.
54
55 4. public Object remove(Object key):
56     --> It will remove a key-value pair from Map on the basis of the
        specified key.
57
58 5. public int size():
59     --> It will return number of key-value pairs of a Map.
60
61 6. public boolean containsKey(Object key):
62     --> It will check whether the specified key is existed or not at
        keys side.
63
64 7. public boolean cotainsValue(Object key):
65     --> It will check whether the specified value is available or not at
        values side.
66
67 8. public Set keySet():
68     --> It will return all keys in the form of a Set.
69
70 9. public Collection values():
71     --> It will return all values in the form of a Collection object.
72
73 10. public boolean isEmpty():
74     --> It will check whether the Map object is empty or not.
75     --> If the present map object is empty then it will return true
        value otherwise it will return false value.
76
77 */
78
79 import java.util.Collection;
80 import java.util.HashMap;
81 import java.util.Set;
82
83 public class Collection10 {
84     public static void main(String[] args) {
85         HashMap<Integer,String> hm1 = new HashMap<>();
86         HashMap hm2 = new HashMap<>(10);
87         HashMap hm3 = new HashMap<>(10, 0.75f);
88
89         // 1. public void put(Object key, Object value):
90
91         hm1.put(1, "CVM");
92         hm1.put(2, "HDS");
93         hm1.put(3, "PAT");
94         //hm1.put(1, "MHM");
95         hm1.put(null, null);
96         hm1.put(4, null);
97         hm1.put(0, "PAT");
98         hm1.put(-9, "VYP");
99         hm1.put(-8, "VYP");
100
101
102         System.out.println(hm1);
103

```

```

104     HashMap hm4 = new HashMap<>(hm1);
105
106     System.out.println(hm4);
107
108     // 2. public void putAll(Map m):
109
110     hm2.putAll(hm3);
111
112     System.out.println(hm2);
113
114     // 3. public Object get(Object key):
115
116     System.out.println(hm1.get("1"));
117
118     // 4. public Object remove(Object key):
119     //     public Object remove (Object key, Object value):
120
121     System.out.println(hm1.remove(1, "ABC"));
122     System.out.println(hm1.remove(2));
123
124     System.out.println(hm1);
125
126     // 5. public int size():
127
128     System.out.println(hm1.size());
129
130     // 6. public boolean containsKey(Object key):
131
132     System.out.println(hm1.containsKey(1));
133     System.out.println(hm1.containsKey(2));
134
135     // 7. public boolean cotainsValue(Object key):
136
137     System.out.println(hm1.containsValue("CVM"));
138     System.out.println(hm1.containsValue("ABC"));
139
140     // 8. public Set keySet():
141
142     Set s = hm1.keySet();
143     System.out.println(s);
144
145     // 9. public Collection values():
146
147     Collection c = hm1.values();
148     System.out.println(c);
149
150     // 10. public boolean isEmpty():
151
152     System.out.println(hm1.isEmpty());
153     System.out.println(hm3.isEmpty());
154 }
155 }

```